

幻尔科技

多模态大模型应用



Hiwonder 官方网站



版本号	修改日期	修改摘要
V1.0	20250809	初次发布

目录

1. 大模型 API-KEY 获取及配置	1
1.1. 阿里云大模型账号注册及 API 密钥获取	1
1.1.1. 注册账号	1
1.1.2. 部署并测试在线大模型	4
1.1.3. API 密钥获取	6
1.2. 阶跃星辰部署及 API 密钥获取	7
1.2.1. 账号注册	8
1.2.2. 密钥获取	11
1.3. API 配置	11
2. 语音控制	12
2.1. 玩法简介	12
2.2. 准备工作	13
2.2.1. 确认 WonderEcho Pro 固件	13
2.2.2. 配置大模型相关 key	13
2.2.3. 配置网络	13
2.3. 玩法步骤	13
2.4. 实现效果	16
2.5. 程序分析	16
2.5.1. launch 文件分析	16
2.5.2. python 文件分析	17
3. 色块分拣	22

3.1. 玩法简介	22
3.2. 准备工作	22
3.2.1. 确认 WonderEcho Pro 固件	22
3.2.2. 配置大模型相关 key	23
3.2.3. 配置网络	23
3.3. 玩法步骤	23
3.4. 实现效果	25
3.5. 程序分析	26
3.5.1. Launch 文件分析	26
3.5.2. Python 文件分析	27
4. 垃圾分类	31
4.1. 玩法简介	32
4.2. 准备工作	32
4.2.1. 确认 WonderEcho Pro 固件	32
4.2.2. 配置大模型相关 key	32
4.2.3. 配置网络	32
4.3. 玩法步骤	33
4.4. 实现效果	35
4.5. 程序分析	35
4.5.1. Launch 文件分析	35
4.5.2. Python 文件分析	37
5. 三维物体分拣	42

5.1. 玩法简介	42
5.2. 准备工作	42
5.2.1. 确认 WonderEcho Pro 固件	42
5.2.2. 配置大模型相关 key	42
5.2.3. 配置网络	42
5.3. 玩法步骤	43
5.4. 实现效果	45
5.5. 程序分析	46
5.5.1. Launch 文件分析	46
5.5.2. Python 文件分析	47

1. 大模型 API-KEY 获取及配置

在大模型的学习过程当中，使用网络大模型就需要获取对应的大模型API KEY, 具体的步骤可以参考以下的内容。

1.1. 阿里云大模型账号注册及 API 密钥获取

1.1.1. 注册账号

1) 复制网址：<https://www.aliyun.com/product/tongyi>，打开通义大模型网页，点击右上角注册。



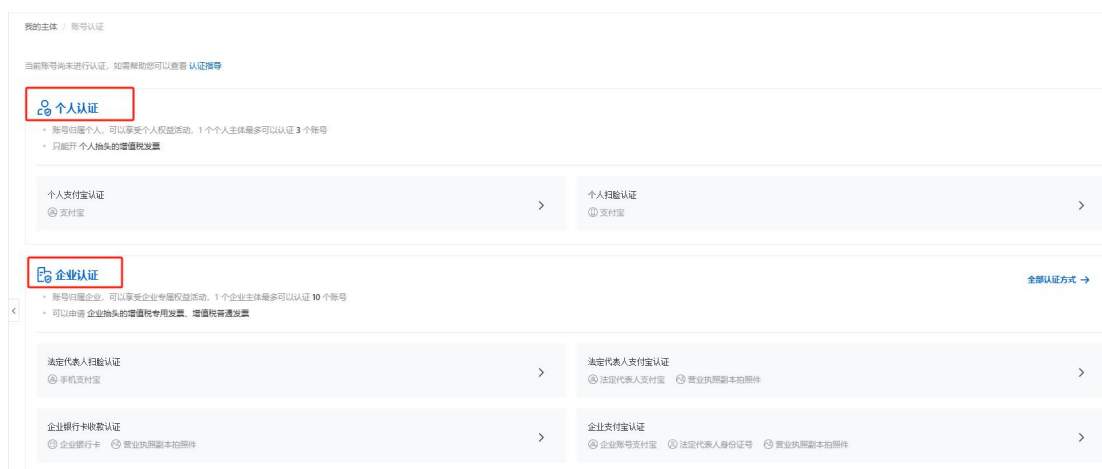
2) 根据提示词依次填写“手机号”、“校验码”，还需要勾选同意“我已阅读并同意用户协议、隐私政策、产品服务协议”。

A screenshot of the Alibaba Cloud registration page. On the left, there is a '扫码注册' (Scan QR code to register) section with a large QR code and text: '使用阿里云APP/支付宝/钉钉 下载阿里云APP, 上云更轻松'. On the right, there is a '手机号注册' (Mobile number registration) section. It has tabs for '手机号注册' (selected) and '账号密码注册' (Account and password registration). Below the tabs, it says '未注册手机号验证通过后将自动注册'. There is a field for '+86 输入手机号' and another for '请输入校验码' with a '获取验证码' (Get verification code) button. At the bottom, there is a checkbox for '我已阅读并同意 用户协议、隐私政策、产品服务协议' and a blue '注册' (Register) button. At the very bottom, a small line of text reads: '登录视为您已同意第三方账号绑定协议、用户协议、隐私政策、产品服务协议'.

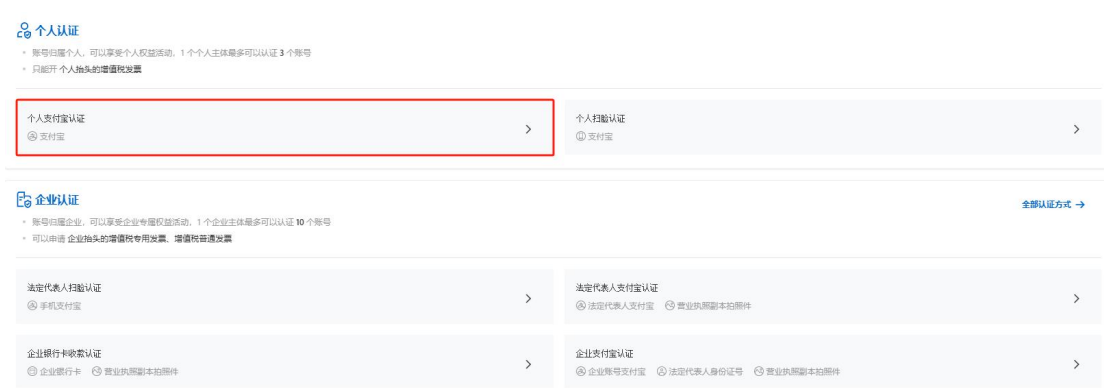
3) 提交完成后，表明账号已注册成功，然后点击“快速实名认证”。



4) 实名认证分为“个人认证”和“企业认证”，可根据实际情况选择对应的认证方式。



5) 本教程以“个人认证”中的“个人支付宝认证”为例。



6) 点击“个人支付宝认证”后，在右边的弹窗中勾选“我同意授权阿里云获取支付宝认证信息”，然后点击“登录支付宝”。



7) 然后在弹出的界面中选择支付宝登录方式。



8) 勾选“已阅读和同意《用户授权协议》”并点击“授权”，即可完成实名认证。

1.1.1.2. 部署并测试在线大模型

1) 注册完成后，点击“免费体验”。



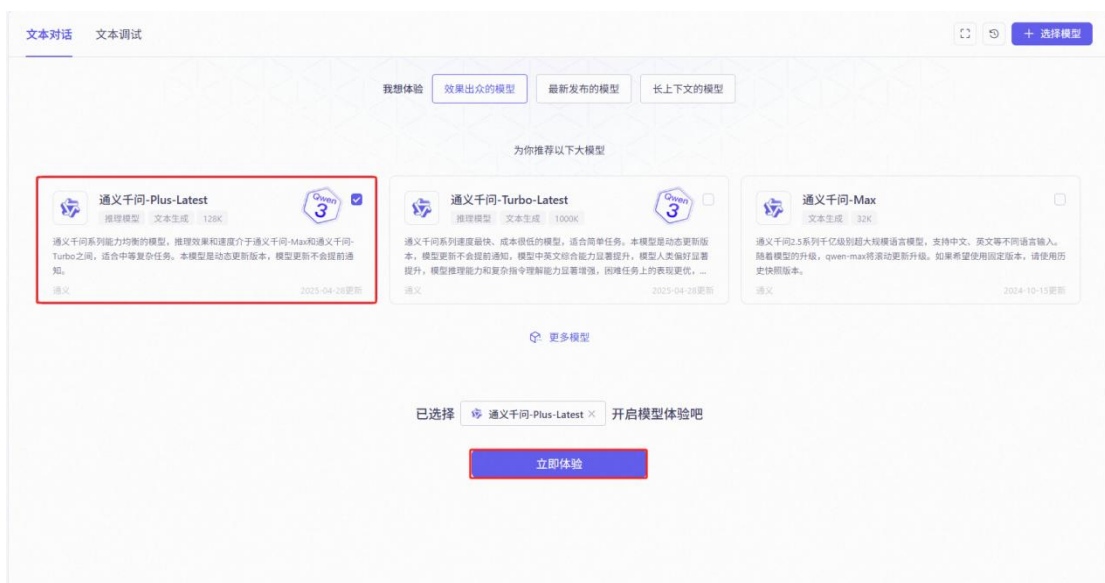
2) 进入大模型界面后，点击网页上方的“开通服务”。



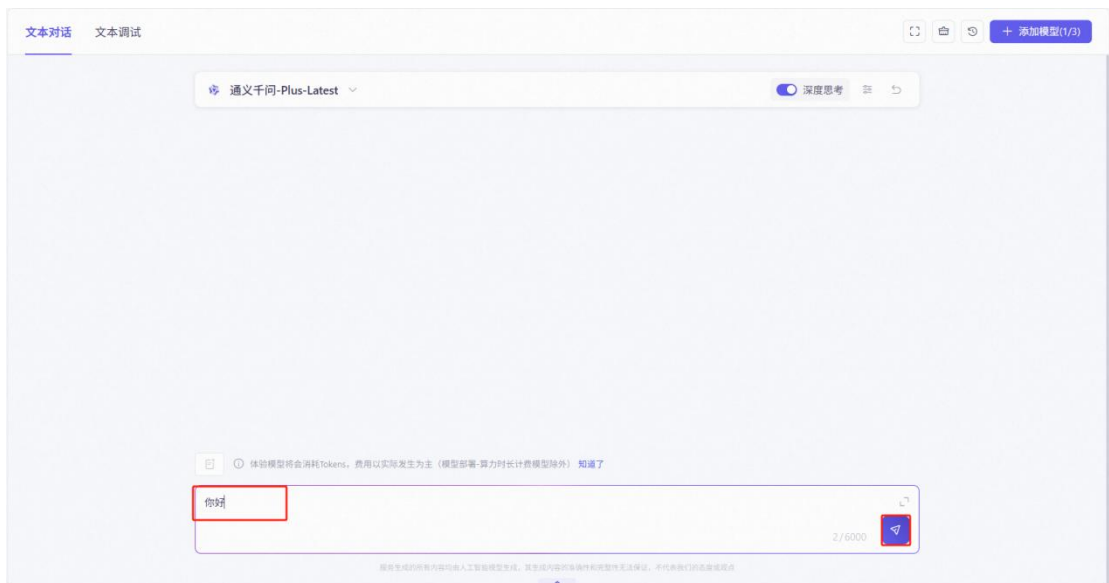
3) 在弹出的界面中，勾选“我已阅读并同意《模型管理服务协议》”，并点击“确认开通，并领取免费额度”。



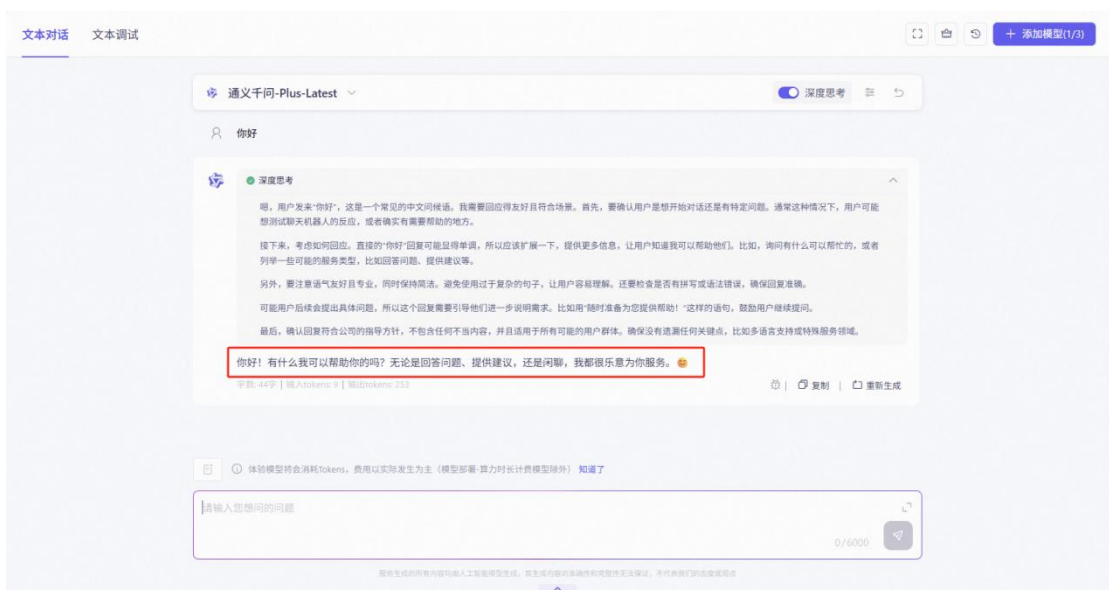
4) 本教程选择了“通义千问-Plus”模型，并点击“立即体验”。



5) 在会话框输入“你好”，并点击“”发送会话。

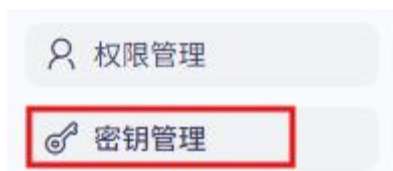


6) 模型生成对话，即可表明账号等配置完成，并通过测试。



1.1.3. API 密钥获取

1) 点击左下角“密钥管理”。



2) 然后点击网页右上角的“创建我的API-KEY”。



3) 在归属业务空间选择“默认业务空间”，描述选项可进行选填，然后点击“确定”，即可创建API-KEY。



4) API KEY 为程序调用的密钥，点击查看后复制（同时需要记住本次生成的 API_KEY, 建议将其保存到本地电脑当中）。

ID	API KEY	所归属业务空间	描述	创建时间	操作
1898058	sk-1****904b	默认业务空间 (llm-5sdh0y2jrzv503my)		2025-03-05 11:58:46	查看 删除

1.2. 阶跃星辰部署及 API 密钥获取

阶跃星辰的提供了丰富的视觉大模型给我们来进行学习和调用, 接下来是获阶跃星辰取key以及配置的方法。

1.2.1. 账号注册

1) 进入阶跃星辰官网。

网址: <https://www.stepfun.com/>



2) 点击右上角的“开放平台”；



3) 点击右上角的“未登录”



4) 使用自己的手机号，获取验证码快捷登录

5) 此时会弹出一个“开发者信息收集”，根据自己的情况进行填写即可，最后点击“提交获取赠送金额”。

开发者信息收集

×

开发者类型

☒ 企业 ☐ 个人

* 公司名称

请输入公司名称

* 公司规模

请输入公司人数

* 所属行业

请选择所属行业

* 产品阶段

请选择产品所处阶段

* 业务场景

请描述业务使用场景

* 申请人姓名

请输入你的姓名

提交获取赠送金额

1.2.2. 密钥获取

1) 接着点击接口密钥，然后在右侧的显示栏中即使我们获取的密钥（注册时平台会设置默认密钥）；



2) 参考下图的箭头，就可以将我们的密钥进行复制（同时建议将密钥保存到自己本地的 PC 电脑当中），后续学习过程当中这个密钥将用于我们大模型相关功能的配置；

接口密钥

密钥最多支持创建10个，请妥善保管
为了保障您的账号安全，我们可能会自动轮换已被泄密的API密钥

名称	密钥	创建时间
默认密钥	3XcujL...dXMqM6	2025-03-18 16:55:36

1.3. API 配置

参考“[大模型 API-KEY 获取及配置](#)”教程，获取大模型的 api_key, 输入以下指令打开配置文件。

```
vim /home/ubuntu/ros2_ws/src/large_models/large_models/config.py
```


2.2. 准备工作

2.2.1. 确认 WonderEcho Pro 固件

WonderEcho Pro 出厂默认的唤醒词是“hello, hiwonder”。如果想要更换唤醒词为“小幻小幻”，可以参考教程目录下“11. 语音控制课程\语音基础课程\02 固件烧录”及同目录下的“固件烧录视频”。

假设之前已经根据以上方法，烧录过其他固件（例如“小幻小幻”唤醒词固件），则需要使用对应的唤醒词进行唤醒。

下面的课程示例默认以出厂唤醒词“hello, hiwonder”为例进行说明。

2.2.2. 配置大模型相关 key

程序默认不配置大模型相关密钥，在开启玩法前需要可以参考本章节“[大模型 API-KEY 获取及配置](#)”完成对相关密钥的配置，此步骤不可跳过，不然影响大模型相关玩法的体验。

2.2.3. 配置网络

注意：本节课的大模型是属于线上大模型，在开始前需要准备一根网线用于连接网络或者将机器人切换成局域网模式。

在此玩法模式下，机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。网络的具体配置方法有以下两种方式：

- 通过手机 APP 进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“手机 APP 控制/连接模式介绍（选看）”。

- 通过 VNC 远程连接到机器人修改相关配置文件进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“开发环境搭建及配置/局域网模式连接”。

2.3. 玩法步骤

 1. 指令输入需严格区分大小写及空格。

2. 机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。

- 1) 点击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。
- 2) 输入指令，关闭手机 app 自启服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

- 3) 输入指令，按下回车键运行语音控制的玩法。

```
ros2 launch large_models llm_control_servo.launch.py
```

```
ros2 launch large_models llm_control_servo.launch.py
```

- 4) 当命令行出现如下图输出并播报“准备就绪”，表示 AI 语音交互盒完成初始化，此时可以说出唤醒词“hello hiwonder”。

```
[tts_node-7] [INFO] [1754290206.584553068] [tts_node]: start
[vocal_detect-5] [INFO] [1754290206.611367567] [vocal_detect]: start
[agent_process-6] [INFO] [1754290206.757935976] [agent_process]: start
[agent_process-6] [INFO] [1754290206.944355533] [agent_process]: set model
[agent_process-6] [INFO] [1754290207.015860352] [agent_process]: set prompt
[llm_control_servo-8] [INFO] [1754290208.474245364] [llm_control_servo]: start
```

- 5) 当命令行出现如下图输出，AI 语音交互盒播报“我在”，表示 AI 语音交互盒已被唤醒激活。此时开始录制用户的指令。

```
[vocal_detect-5] [INFO] [1754042185.692041748] [vocal_detect]: asr...
[vocal_detect-5] [INFO] [1754042185.723628866] [vocal_detect]: Recording.....
```

- 6) 我们可以根据程序出现的**任务示例**里面的**输入**提示信息说出命令。

```
[llm_control_servo-8] ## 动作函数库
[llm_control_servo-8] - 控制舵机在指定时间转到指定位置：set_position(1, ((1, -120), (2, 90)))
[llm_control_servo-8] - 控制机械臂往指定方向移动一段距离：move(x, y, z)
[llm_control_servo-8] - 延时指定时间：time.sleep(1)
[llm_control_servo-8] ## 任务示例
[llm_control_servo-8] 输入：1号舵机，2号舵机在2秒内分别转到-90度和90度
[llm_control_servo-8] 输出：{'action':['set_position(2, [[1, -90], [2, 90]])'], 'response': '收到，马上执行'}
[llm_control_servo-8] 输入：1号舵机在1秒内转到45度
[llm_control_servo-8] 输出：{'action':['set_position(1, [[1, 45]])'], 'response': '好的'}
[llm_control_servo-8] 输入：6号舵机转到15度
[llm_control_servo-8] 输出：{'action':['set_position(1, [[6, 15]])'], 'response': '好的，6号舵机转到15度'}
[llm_control_servo-8] 输入：3号舵机转到11度，4号舵机转到0度，停2s
[llm_control_servo-8] 输出：{'action':['set_position(1, [[3, 11], [4, 0]])'], 'time.sleep(2)'], 'response': '好啦'}
[llm_control_servo-8] 输入：1号舵机转到20度，3号舵机转到45度，停1s，1号舵机转到50度，2号舵机转到0度
[llm_control_servo-8] 输出：{'action':['set_position(1, [[1, 20], [3, 45]])'], 'time.sleep(1)', 'set_position(1, [[2, 0]])'], 'response': '好啦'}
[llm_control_servo-8]
```

7) 当命令行出现如下图输出，表示 **AI 语音交互盒** 将识别到的语音打印出来。

```
[vocal_detect-7] [INFO] [1741419609.381474185] [vocal_detect]: 以
[vocal_detect-7] [INFO] [1741419609.116649226] [vocal_detect]: 一号
[vocal_detect-7] [INFO] [1741419609.607889811] [vocal_detect]: 一号舵机
[vocal_detect-7] [INFO] [1741419610.077837835] [vocal_detect]: 一号舵机旋转9
[vocal_detect-7] [INFO] [1741419611.718256598] [vocal_detect]: 一号舵机旋转90度。
```

8) 当命令行出现如下图输出，表示成功调用云端语音大模型的语音识别服务，对用户指令音频进行解析，“**publish_asr_result**”中为解析结果。

```
[vocal_detect-5] [INFO] [1754290448.698139155] [vocal_detect]: Done recording
[vocal_detect-5] [INFO] [1754290449.314853122] [vocal_detect]: publish asr result:一号舵机旋转90度。
```

9) 当命令行出现如下图输出，表示成功调用云端大语言大模型，对用户指令进行思考，并给出语言回答“**response**”并设计符合用户指令语义的动作。

 回答为大模型自动生成，只保证回复的语义准确，回复的文字编排具有随机性。

```
[agent_process-6] [INFO] [1754290449.315102547] [agent_process]: thinking...
[agent_process-6] [INFO] [1754290451.387034090] [agent_process]: publish llm result: {'action': ['set_position(1, [[1, 90]])'], 'response': '一号舵机旋转90度，完成指令！'}
[agent_process-6]
```

10) 当命令行出现如下图输出，表示一轮对话交互已经结束，您可以参考步骤 4)，重新以唤醒词唤醒 **AI 语音交互盒**，开始新一轮对话交互。

```
[tts_node-7] [INFO] [1754290451.531168311] [tts_node]: Speech synthesizer is opened.
[tts_node-7] [INFO] [1754290454.440536289] [tts_node]: Speech synthesizer is completed.
[tts_node-7] [INFO] [1754290454.442373689] [tts_node]: Speech synthesizer is closed.
[vocal_detect-5] [INFO] [1754290454.454620413] [vocal_detect]: enable_wakeup
```

11) 如需关闭此玩法，只需要在终端界面中按下“**Ctrl+C**”。如果关闭失败，可多次按下。

2.4. 实现效果

玩法开启后，我们可以自由组织语言，例如：“1 号舵机旋转 90°”，控制 ArmPi Ultra 的 1 号舵机旋转 90° 方向。

2.5. 程序分析

2.5.1. launch 文件分析

程序路径：

/home/ubuntu/ros2_ws/src/large_models/large_models/llm_control_servo.

launch.py

◆ launch_setup函数

```
def launch_setup(context):
    mode = LaunchConfiguration('mode', default=1)
    mode_arg = DeclareLaunchArgument('mode', default_value=mode)

    sdk_package_path = get_package_share_directory('sdk')
    large_models_package_path = get_package_share_directory('large_models')

    sdk_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(sdk_package_path, 'launch/arm_pi_ultra.launch.py')),
    )

    large_models_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(large_models_package_path, 'launch/start.launch.py')),
        launch_arguments={ 'mode': mode }.items(),
    )

    llm_control_servo_node = Node(
        package='large_models',
        executable='llm_control_servo',
        output='screen',
    )

    return [mode_arg,
            sdk_launch,
            large_models_launch,
            llm_control_servo_node,
    ]
```

加载 sdk 功能包的 `launch/arm_pi_ultra.launch.py`, 启动机械臂底层控制服务（如舵机驱动、运动学计算），为后续控制提供硬件支持。加载 `large_models` 功能包的 `launch/start.launch.py`, 启动大模型相关服务，并通过 `launch_arguments` 将 `mode` 参数传入，控制大模型运行模式（如不同交互逻辑或功能开关）。声明 `llm_control_servo_node`, 启动 `large_models` 功能包中的 `llm_control_servo` 可执行文件，核心功能是将大模型的输出转换为机械臂控制指令，实现“大模型决策→机械臂动作”的闭环，日志输出到屏幕。

◆ `generate_launch_description` 函数

```
def generate_launch_description():  
    return LaunchDescription([  
        OpaqueFunction(function = launch_setup)  
    ])
```

创建并返回 `LaunchDescription` 对象，通过 `OpaqueFunction` 调用 `launch_setup` 函数，作为 ROS 2 启动文件的标准入口。

◆ 主函数

```
if __name__ == '__main__':  
    # 创建一个LaunchDescription对象  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 `LaunchDescription` 对象和 `LaunchService` 服务，将启动描述添加到启动服务并运行，实现手动启动整个系统的功能

2.5.2. python 文件分析

程序路径：

```
/home/ubuntu/ros2_ws/src/large_models/large_models/llm_control_servo.  
py
```

◆ 定义提示词模板（PROMPT）

```
if os.environ["ASR_LANGUAGE"] == 'Chinese':
    PROMPT = '''
#角色任务
你是一款智能机械臂，需要根据输入的内容，生成对应的指令。

##要求与限制
1.根据输入的动作内容，在动作函数库中找到对应的指令，并输出对应的指令。
2.为每个动作序列编织一句精炼（10至30字）、风趣且变化无穷的反馈信息，让交流过程妙趣横生。
3.上为z轴正，左为y轴负，前为x轴正，单位米
4.舵机编号为1,2,3,4,5,6，转动范围为[-120, 120]度，如果超过了范围action就返回[]
5.初始位置[0, 48, -72, -96, 0, -30]
6.直接输出json结果，不要分析，不要输出多余内容。
7.格式：{'action':['xx', "xx"], 'response':'xx'}

##结构要求##
- ``"action"``键下承载一个按执行顺序排列的函数名称字符串数组，当找不到对应动作函数时action输出[]。
- ``"response"``键则配以精心构思的简短回复，完美贴合上述字数与风格要求。
- **特殊处理**：如果没有指定舵机运行时间就固定为1秒，如果两个动作间没有延时或者停顿就把他们合一起

## 动作函数库
- 控制舵机在指定时间转到指定位置：set_position(1, ((1, -120), (2, 90)))
- 控制机械臂往指定方向移动一段距离：move(x, y, z)
- 延时指定时间：time.sleep(1)

## 任务示例
输入：1号舵机，2号舵机在2秒内分别转到-90度和90度
输出：{'action':['set_position(2, [[1, -90], [2, 90]])'], 'response':'收到，马上执行'}
输入：1号舵机在1秒内转到45度
输出：{'action':['set_position(1, [[1, 45]])'], 'response':'好的'}
输入：6号舵机转到15度
输出：{'action':['set_position(1, [[6, 15]])'], 'response':'好的，6号舵机转到15度'}
输入：3号舵机转到11度，4号舵机转到0度，停2s
输出：{'action':['set_position(1, [[3, 11], [4, 0]])', 'time.sleep(2)'], 'response':'好嘞'}
输入：1号舵机转到20度，3号舵机转到45度，停1s，1号舵机转到50度，2号舵机转到0度
输出：{'action':['set_position(1, [[1, 20], [3, 45]])', 'time.sleep(1)', 'set_position(1, [[2, 0]])'], 'response':'好嘞'}
```

◆ 初始化变量

```
class LLMControlServo(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name)

        self.action = []
        self.interrupt = False
        self.llm_result = ''
        self.action_finish = False
        self.play_audio_finish = False
        # self.llm_result = '{\'action\': [\'set_position(1, [[10, -20]])\'], \'response\': \'好嘞\'}'
        # self.llm_result = '{\'action\': [\'move(0, 0, -0.05)\'], \'response\': \'坐稳了，准备下移5cm! \'}'
        self.running = True
        self.id_dict = {'6': 0, "5": 1, "4": 2, "3": 3, "2": 4, "1": 5}
        self.current_pulse = [500, 600, 825, 110, 500, 210]

        self.current_position = [0.1, 0, 0.226]

        timer_cb_group = ReentrantCallbackGroup()
        self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
        self.create_subscription(String, 'agent_process/result', self.llm_result_callback, 1)
        self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_finish_callback, 1, callback_group=timer_cb_group)
        self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wakeup')
        self.awake_client.wait_for_service()
        self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callback, 1)

        self.set_model_client = self.create_client(SetModel, 'agent_process/set_model')
        self.set_model_client.wait_for_service()
        self.set_prompt_client = self.create_client(SetString, 'agent_process/set_prompt')
        self.set_prompt_client.wait_for_service()
        self.set_pose_target_client = self.create_client(SetRobotPose, 'kinematics/set_pose_target')
        self.set_pose_target_client.wait_for_service()
        self.set_joint_value_target_client = self.create_client(SetJointValue, 'kinematics/set_joint_value_target')
        self.set_joint_value_target_client.wait_for_service()
        self.joints_pub = self.create_publisher(ServosPosition, 'servo_controller', 1) # Servo control 舵机控制

        self.timer = self.create_timer(0.0, self.init_process, callback_group=timer_cb_group)
```

设置状态变量（如动作列表action、中断标志interrupt等），记录当前舵机位置和机械臂位姿。创建发布者（如 TTS 文本发布）、订阅者（如大模型结果、TTS 播放完成信号、唤醒信号）。创建服务客户端（如设置大模型、舵机控制、机械臂位姿控制等），用于通信和控制。

◆ get_node_state方法

```
def get_node_state(self, request, response):  
    return response
```

服务回调函数，返回空响应，用于节点状态查询（如初始化完成确认）。

◆ init_process方法

```
def init_process(self):  
    self.timer.cancel()  
  
    msg = SetModel.Request()  
    msg.model = llm_model  
    msg.model_type = 'llm'  
    msg.api_key = api_key  
    msg.base_url = base_url  
    self.send_request(self.set_model_client, msg)  
  
    msg = SetString.Request()  
    msg.data = PROMPT  
    self.send_request(self.set_prompt_client, msg)  
    # -92  
    self.set_position(1.5, [[6, 0], [5, 48], [4, 72], [3, -96], [2, 0], [1, -30]])  
    msg = set_joint_value_target(self.current_pulse[:-1])  
    endpoint = self.send_request(self.set_joint_value_target_client, msg)  
    pose_t, pose_r = endpoint.pose.position, endpoint.pose.orientation  
    mat = common.xyz_quat_to_mat([pose_t.x, pose_t.y, pose_t.z], [pose_r.w, pose_r.x, pose_r.y, pose_r.z])  
    position, rpy = common.mat_to_xyz_euler(mat)  
    self.current_position = [position, rpy]  
  
    speech.play_audio(start_audio_path)  
    threading.Thread(target=self.process, daemon=True).start()  
    self.create_service(Empty, '~/init_finish', self.get_node_state)  
    self.get_logger().info('\033[1;32m%s\033[0m' % 'start')  
    self.get_logger().info('\033[1;32m%s\033[0m' % PROMPT)
```

◆ send_request方法

```
def send_request(self, client, msg):  
    future = client.call_async(msg)  
    while rclpy.ok():  
        if future.done() and future.result():  
            return future.result()
```

通用服务调用工具，发送服务请求并阻塞等待结果返回，确保服务通信完成（如设置大

模型、控制机械臂位姿）。

◆ wakeup_callback方法

```
def wakeup_callback(self, msg):
    if self.llm_result:
        self.get_logger().info('wakeup interrupt')
        self.interrupt = msg.data
```

唤醒信号订阅回调，当收到唤醒信号（如语音唤醒）时，设置 interrupt 标志，用于中断当前执行的动作。

◆ llm_result_callback方法

```
def llm_result_callback(self, msg):
    self.llm_result = msg.data
```

大模型结果订阅回调，接收大模型生成的指令（含 action 和 response），保存到 llm_result 变量，供后续处理。

◆ set_position方法

```
def set_position(self, duration, positions):
    msg = ServosPosition()
    msg.duration = float(duration)
    position_list = []
    for i in positions:
        position = ServoPosition()
        position.id = i[0]
        position.position = float(i[1])
        position_list.append(position)
        if i[0] != 10:
            self.current_pulse[self.id_dict[str(i[0])]] = 500 + float(i[1])/240*1000
        else:
            self.current_pulse[self.id_dict[str(i[0])]] = 700 + float(i[1])/240*1000
    msg.position = position_list
    msg.position_unit = "deg"
    self.joints_pub.publish(msg)
    # self.get_logger().info(f'{self.current_pulse}')
    msg = set_joint_value_target(self.current_pulse[:-1])
    endpoint = self.send_request(self.set_joint_value_target_client, msg)
    pose_t, pose_r = endpoint.pose.position, endpoint.pose.orientation
    mat = common.xyz_quat_to_mat([pose_t.x, pose_t.y, pose_t.z], [pose_r.w, pose_r.x, pose_r.y, pose_r.z])
    position, rpy = common.mat_to_xyz_euler(mat)
    self.current_position = [position, rpy]
    # self.get_logger().info(f'{self.current_position}')
```

根据输入的时间和目标位置，发布舵机控制消息（ServosPosition），更新当前舵机位置记录，并同步机械臂关节位姿。

◆ move方法

```
def move(self, x, y, z):
    self.current_position[0][0] += x
    self.current_position[0][1] += y
    self.current_position[0][2] += z
    msg = set_pose_target(self.current_position[0], self.current_position[1][1], [-180.0, 180.0], 1.0)
    res = self.send_request(self.set_pose_target_client, msg)
    if res.pulse:
        servo_data = res.pulse
        set_servo_position(self.joints_pub, 1, ((4, servo_data[3]), (3, servo_data[2]), (2, servo_data[1]), (1, servo_data[0])))
        self.current_pulse[:-1] = servo_data
```

◆ play_audio_finish_callback方法

```
def play_audio_finish_callback(self, msg):
    self.play_audio_finish = msg.data
```

处理语音播放完成后的回调，重新启用语音唤醒功能。

◆ process方法

```
def process(self):
    while self.running:
        if self.llm_result:
            msg = String()
            if 'action' in self.llm_result: # If a corresponding action is returned, extract it and proceed with processing. 如果有对应的行为返回那么就提取处理
                result = eval(self.llm_result[self.llm_result.find('{'):self.llm_result.find('}') + 1])
                # self.get_logger().info(str(result))
                action_list = []
                if 'action' in result:
                    action_list = result['action']
                if 'response' in result:
                    response = result['response']
                msg.data = response
                self.tts_text_pub.publish(msg)
                for i in action_list:
                    if self.interrupt:
                        self.get_logger().info('interrupt')
                        break
                    if 'set_position' in i:
                        eval('self.' + i)
                    elif 'move' in i:
                        eval('self.' + i)
                    else:
                        eval(i)
                else: # No corresponding action, just respond. 没有对应的行为，只回答
                    response = self.llm_result
                    msg.data = response
                    self.tts_text_pub.publish(msg)
                    self.action_finish = True
                    self.llm_result = ''
            else:
                time.sleep(0.01)
            if self.play_audio_finish and self.action_finish:
                self.play_audio_finish = False
                self.action_finish = False
                msg = SetBool.Request()
                msg.data = True
                self.send_request(self.awake_client, msg)
    rclpy.shutdown()
```

循环检查 llm_result（大模型输出），解析出动作列表（action）和回复文本（response）。发布回复文本到 TTS 节点（语音播放），依次执行动作（如 set_position 控制舵机、move 移动机械臂、time.sleep 延时）。动作和语音完成后，重新启用唤醒功能，等待下一次指令。

◆ main函数

```
def main():
    node = LLMControlServo('llm_control_servo')
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    executor.spin()
    node.destroy_node()
```

创建 LLMControlServo 节点实例，启动多线程执行器处理回调，运行节点并处理中断（如键盘终止），确保资源清理。

3. 色块分拣

本节课的大模型属于线上大模型，需要让我们使用的主控保持网络通畅（建议使用有线网络进行连接）。

3.1. 玩法简介

程序开始运行，“AI 语音交互盒”播报“准备就绪”后。说出指定的唤醒词“**hello hiwonder**”激活“AI 语音交互盒”，“AI 语音交互盒”会回应“我在”。然后即可通过语音控制 ArmPi Ultra，例如“把红色的方块拿走”，终端会打印出识别到的语音，“AI 语音交互盒”会播报思考后生成的回答，并执行对应的玩法。

3.2. 准备工作

3.2.1. 确认 WonderEcho Pro 固件

WonderEcho Pro 出厂默认的唤醒词是“**hello, hiwonder**”。如果想要更换唤醒词为“**小幻小幻**”，可以参考教程目录下“**11. 语音控制课程\语音基础课程\02 固件烧录**”及同目录下的“**固件烧录视频**”。

假设之前已经根据以上方法，烧录过其他固件（例如“**小幻小幻**”唤醒词固件），则需要使用对应的唤醒词进行唤醒。

下面的课程示例默认以出厂唤醒词“**hello, hiwonder**”为例进行说明。

3.2.2. 配置大模型相关 key

程序默认不配置大模型相关密钥，在开启玩法前需要可以参考本章节“[大模型 API-KEY 获取及配置](#)”完成对相关密钥的配置，此步骤不可跳过，不然影响大模型相关玩法的体验。

3.2.3. 配置网络

注意:本节课的大模型是属于线上大模型，在开始前需要准备一根网线用于连接网络或者将机器人切换成局域网模式。

在此玩法模式下，机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。网络的具体配置方法有以下两种方式：

- 通过手机 APP 进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“手机 APP 控制/连接模式介绍（选看）”。

- 通过 VNC 远程连接到机器人修改相关配置文件进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“开发环境搭建及配置/局域网模式连接”。

3.3. 玩法步骤

i 1. 指令输入需严格区分大小写及空格。

2. 机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。

- 1) 点击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。
- 2) 输入指令，关闭手机 app 自启服务。

```
~/.stop_ros.sh
```

```
> ~/.stop_ros.sh
```

- 3) 输入指令，按下回车键运行色块分拣的玩法。

```
ros2 launch large_models llm_object_sorting.launch.py
```

```
ros2 launch large_models llm_object_sorting.launch.py
```

4) 当命令行出现如下图输出并播报“准备就绪”，表示“AI 语音交互盒”完成初始化，此时可以说出唤醒词“hello hiwonder”。

```
[tts_node-11] [INFO] [1754295914.467975407] [tts_node]: start
[vocal_detect-9] [INFO] [1754295914.486351903] [vocal_detect]: start
[agent_process-10] [INFO] [1754295914.633483235] [agent_process]: start
[agent_process-10] [INFO] [1754295914.712734745] [agent_process]: set model
[agent_process-10] [INFO] [1754295914.790219800] [agent_process]: set prompt
[object_sorting-8] [INFO] [1754295914.817218882] [object_sorting]: enter object sorting
[llm_object_sorting-12] [INFO] [1754295916.296307634] [llm_color_sorting]: start
```

5) 当命令行出现如下图输出，“AI 语音交互盒”播报“我在”，表示“AI 语音交互盒”已被唤醒激活。此时开始录制用户的指令。

```
[vocal_detect-9] [INFO] [1754297969.419809848] [vocal_detect]: asr...
[vocal_detect-9] [INFO] [1754297969.471216506] [vocal_detect]: Recording.....
```

6) 我们可以根据程序出现的任务示例里面的输入提示信息说出命令。

```
[llm_object_sorting-12] ## 动作函数库
[llm_object_sorting-12] - 拿起不同颜色的物体：color_sorting('red')
[llm_object_sorting-12]
[llm_object_sorting-12] ## 任务示例
[llm_object_sorting-12] 输入：拿走绿色方块
[llm_object_sorting-12] 输出：{'action': "color_sorting('green')", 'response': '好的，开始分拣绿色'}
[llm_object_sorting-12] 输入：将蓝色方块拿走
[llm_object_sorting-12] 输出：{'action': "color_sorting('blue')", 'response': '蓝色方块走你'}
[llm_object_sorting-12] 输入：只留下红色，其他拿走
[llm_object_sorting-12] 输出：{'action': "color_sorting('blue', 'green')", 'response': '好的，马上执行'}
[llm_object_sorting-12] 输入：分拣色块
[llm_object_sorting-12] 输出：{'action': "color_sorting('red', 'blue', 'green')", 'response': '没问题'}
```

7) 例如，我们将色块摆放到摄像头的面前，说出口令“将红色的色块拿走”。



8) 当命令行出现如下图输出，表示“AI 语音交互盒”将识别到的语音打印出来。

```
[vocal_detect-9] [INFO] [1754297969.419809848] [vocal_detect]: asr...
[vocal_detect-9] [INFO] [1754297969.471216506] [vocal_detect]: Recording.....
[vocal_detect-9] [INFO] [1754297971.311092030] [vocal_detect]: 讲
[vocal_detect-9] [INFO] [1754297972.358455238] [vocal_detect]: 姜红色的
[vocal_detect-9] [INFO] [1754297974.752432874] [vocal_detect]: 姜红色的方块拿
[vocal_detect-9] [INFO] [1754297974.978121540] [vocal_detect]: 将红色的方块拿走。
[vocal_detect-9] [INFO] [1754297974.989918992] [vocal_detect]: RecognitionCallback sentence e
nd, request_id:3375f96ae77b4ab8abe04971cae9262c, usage:{'duration': 2}
```

9) 当命令行出现如下图输出,表示成功调用云端语音大模型的语音识别服务,对用户指令音频进行解析,“publish_asr_result”中为解析结果。

```
[vocal_detect-9] [INFO] [1754297975.078555826] [vocal_detect]: RecognitionCallback completed.
[vocal_detect-9] [INFO] [1754297975.148417330] [vocal_detect]: Done recording
[vocal_detect-9] [INFO] [1754297975.832286275] [vocal_detect]: publish asr result:将红色的方块拿走。
[agent_process-10] [INFO] [1754297975.838621355] [agent_process]: thinking...
```

10) 当命令行出现如下图输出,表示成功调用云端大语言大模型,对用户指令就行思考,并给出语言回答“response”并设计符合用户指令语义的动作。

 回答为大模型自动生成,只保证回复的语义准确,回复的文字编排具有随机性。

```
[agent_process-10] {'action': "color_sorting('red')", 'response': '红色方块,这就拿走!'}
[agent_process-10] ''
[llm_object_sorting-12] [INFO] [1754297979.625285045] [llm_color_sorting]: ['red']
```

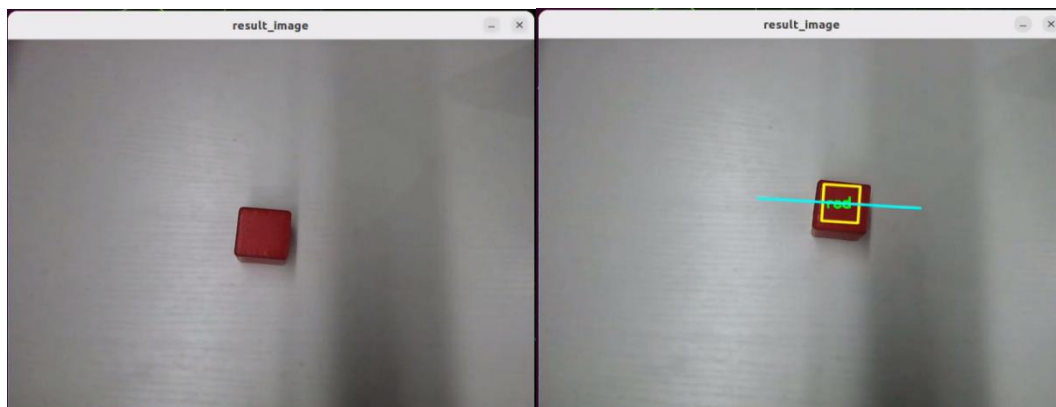
11) 当命令行出现如下图输出,表示一轮对话交互已经结束,您可以参考步骤 4),重新以唤醒词唤醒“AI 语音交互盒”,开始新一轮对话交互。

```
[tts_node-11] [INFO] [1754297982.335247348] [tts_node]: Speech synthesizer is completed.
[tts_node-11] [INFO] [1754297982.350246719] [tts_node]: Speech synthesizer is closed.
[vocal_detect-9] [INFO] [1754297982.368761084] [vocal_detect]: enable_wakeup
```

12) 如需关闭此玩法,只需要在终端界面中按下“Ctrl+C”。如果关闭失败,可多次按下。

3.4. 实现效果

玩法开启后,我们可以自由组织语言,例如:“将红色的方块拿走”,控制 ArmPi Ultra 自动抓取前方视线范围内的绿色方块并放置到机械臂的左边。



3.5. 程序分析

3.5.1. Launch 文件分析

程序路径：

`/home/ubuntu/ros2_ws/src/large_models/large_models/llm_object_sorting.launch.py`

◆ launch_setup函数

```
def launch_setup(context):
    mode = LaunchConfiguration('mode', default=1)
    mode_arg = DeclareLaunchArgument('mode', default_value=mode)

    large_models_package_path = get_package_share_directory('large_models')

    # 立即启动 object_sorting_launch
    object_sorting_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(large_models_package_path, 'launch/object_sorting_node.launch.py'),
        )
    )

    # 在5秒后启动 large_models_launch 和 llm_object_sorting_node
    delayed_launch = TimerAction(
        period=5.0, # 延迟时间为5秒
        actions=[
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(
                    os.path.join(large_models_package_path, 'launch/start.launch.py'),
                    launch_arguments={'mode': mode}.items(),
                ),
            ),
            Node(
                package='large_models',
                executable='llm_object_sorting',
                output='screen',
            ),
        ]
    )

    return [mode_arg,
            object_sorting_launch, # 立即启动
            delayed_launch,       # 5秒后启动
    ]
```

立即加载 large_models 功能包的 launch/object_sorting_node.launch.py，启动物体分拣的基础节点（如物体识别、分类逻辑等核心功能），作为系统的基础组件。通过 TimerAction 设置 5 秒延迟，确保基础节点初始化完成后，再启动依赖组件，加载 large_models 功能包的 launch/start.launch.py，启动大模型服务，并传入 mode 参数控制其运行模式。启动 large_models 功能包中的 llm_object_sorting 可执行文件，作为大模型与物体分拣逻辑的桥梁，将大模型决策转换为分拣动作指令，日志输出到屏幕。

◆ generate_launch_description 函数

```
def generate_launch_description():  
    return LaunchDescription([  
        OpaqueFunction(function = launch_setup)  
    ])
```

创建并返回 LaunchDescription 对象，通过 OpaqueFunction 调用 launch_setup 函数，作为 ROS 2 启动文件的标准入口。

◆ 主函数

```
if __name__ == '__main__':  
    # 创建一个LaunchDescription对象  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务，将启动描述添加到启动服务并运行，实现手动启动整个系统的功能

3.5.2. Python 文件分析

程序路径：

/home/ubuntu/ros2_ws/src/large_models/large_models/llm_object_sorting.py

◆ 定义提示词模板（PROMPT）

```

#角色任务
你是一款智能机械臂，需要根据输入的内容，生成对应的指令。

##要求与限制
1.根据输入的动作内容，在动作函数库中找到对应的指令，并输出对应的指令。
2.为动作序列编织一句精炼（10至30字）、风趣且变化无穷的反馈信息，让交流过程妙趣横生。
3.直接输出json结果，不要分析，不要输出多余内容。
4.可以分拣三种颜色物体：红色（red），绿色（green）和蓝色（blue）
4.格式：{'action':'xx', 'response':'xx'}

##结构要求##
- ``"action"``键下承载一个函数名称字符串数组，当找不到对应动作函数时action输出""。
- ``"response"``键则配以精心构思的简短回复，完美贴合上述字数与风格要求。

## 实施原则 - 在最终输出前，实施全面校验，确保回复不仅格式合规，而且内容充满意趣，无一遗漏上述规范。

## 动作函数库
- 拿起不同颜色的物体：color_sorting('red')

## 任务示例
输入：拿走绿色方块
输出：{'action': "color_sorting('green')", 'response': '好的，开始分拣绿色'}
输入：将蓝色方块拿走
输出：{'action': "color_sorting('blue')", 'response': '蓝色方块走你'}
输入：只留下红色，其他拿走
输出：{'action': "color_sorting('blue','green')", 'response': '好的，马上执行'}
输入：分拣色块
输出：{'action': "color_sorting('red', 'blue','green')", 'response': '没问题'}
'''

```

◆ 初始化变量

```

class LLMColorSorting(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name)

        self.action = []
        self.llm_result = ''
        self.action_finish = False
        self.play_audio_finish = False
        # self.llm_result = '{"action": "color_sorting(\'blue\', \'red\', \'green\', \'tag_1\')", "response": "ok! "}'
        self.running = True

        timer_cb_group = ReentrantCallbackGroup()
        self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
        self.create_subscription(String, 'agent_process/result', self.llm_result_callback, 1)
        self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_finish_callback, 1, callback_group=timer_cb_group)
        self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callback, 1)
        self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wakeup')
        self.awake_client.wait_for_service()

        self.set_model_client = self.create_client(SetModel, 'agent_process/set_model')
        self.set_model_client.wait_for_service()
        self.set_prompt_client = self.create_client(SetString, 'agent_process/set_prompt')
        self.set_prompt_client.wait_for_service()
        self.enter_client = self.create_client(Trigger, 'object_sorting/enter')
        self.enter_client.wait_for_service()
        self.start_client = self.create_client(SetBool, 'object_sorting/enable_sorting')
        self.start_client.wait_for_service()

        self.set_target_client = self.create_client(SetStringBool, 'object_sorting/set_target')
        self.set_target_client.wait_for_service()
        self.timer = self.create_timer(0.0, self.init_process, callback_group=timer_cb_group)

```

初始化节点，配置状态变量（如动作列表、大模型结果等），创建发布者（TTS 文本）、订阅者（大模型结果、TTS 完成信号、唤醒信号）和服务客户端（大模型配置、分拣控制等），启动初始化定时器。

◆ get_node_state方法

```
def get_node_state(self, request, response):  
    return response
```

服务回调函数，返回空响应，用于节点状态查询（如初始化完成确认）。

◆ init_process方法

```
def init_process(self):  
    self.timer.cancel()  
  
    msg = SetModel.Request()  
    msg.model = llm_model  
    msg.model_type = 'llm'  
    msg.api_key = api_key  
    msg.base_url = base_url  
    self.send_request(self.set_model_client, msg)  
  
    msg = SetString.Request()  
    msg.data = PROMPT  
    self.send_request(self.set_prompt_client, msg)  
  
    self.send_request(self.enter_client, Trigger.Request())  
    speech.play_audio(start_audio_path)  
    threading.Thread(target=self.process, daemon=True).start()  
    self.create_service(Empty, '~/init_finish', self.get_node_state)  
    self.get_logger().info('\033[1;32m%s\033[0m' % 'start')  
    self.get_logger().info('\033[1;32m%s\033[0m' % PROMPT)
```

节点初始化流程，取消定时器，配置大模型参数（模型类型、API 密钥等），发送进入分拣服务请求，启动处理线程，发布初始化完成日志。

◆ send_request方法

```
def send_request(self, client, msg):  
    future = client.call_async(msg)  
    while rclpy.ok():  
        if future.done() and future.result():  
            return future.result()
```

通用服务调用工具，发送服务请求并阻塞等待结果，确保服务通信完成（如配置大模型、控制分拣）。

◆ wakeup_callback方法

```
def wakeup_callback(self, msg):  
    if msg.data and self.llm_result:  
        self.send_request(self.enter_client, Trigger.Request())
```

唤醒信号订阅回调，收到唤醒信号时重新发送进入分拣服务请求，重置分拣状态。

◆ llm_result_callback方法

```
def llm_result_callback(self, msg):  
    self.llm_result = msg.data
```

大模型结果订阅回调，接收大模型生成的指令（含 action 和 response），保存到 llm_result 变量，供后续处理。

◆ play_audio_finish_callback方法

```
def play_audio_finish_callback(self, msg):  
    self.play_audio_finish = msg.data
```

TTS 播放完成信号订阅回调，设置 play_audio_finish 标志，同步动作执行与语音播放。

◆ process方法

```

def process(self):
    while self.running:
        if self.llm_result:
            msg = String()
            if 'action' in self.llm_result: # If a corresponding action is returned, extract it and proceed with processing. 如果有对应的行为返回那么就提取处理
                result = eval(self.llm_result[self.llm_result.find('('):self.llm_result.find(')') + 1])
                # result = json.loads(self.llm_result)
                if 'action' in result:
                    text = result['action']
                    # Use regular expressions to extract all strings inside. 使用正则表达式提取括号中的所有字符串
                    pattern = r"color_sorting\(\s*['\"](?:[^\"']*|'(?=[^']*|'')'')*\s*\)"
                    # Use re.search to find the matching result. 使用re.search找到匹配的结果
                    match = re.search(pattern, text)
                    # Extract the result. 提取结果
                    if match:
                        # Retrieve all argument parts (the content inside the parentheses). 获取所有的参数部分 (括号内的内容)
                        params = match.group(1)
                        # Use regular expressions to split the string arguments. 使用正则拆分成各个字符串参数
                        colors = re.findall(r"['\"](?:[^\"']*|'(?=[^']*|'')'')*", params)
                        self.get_logger().info(str(colors))
                        color_msg = SetStringBool.Request()
                        for i in ['red', 'green', 'blue', 'tag_1', 'tag_2', 'tag_3']:
                            color_msg.data_str = i
                            if i in colors:
                                color_msg.data_bool = True
                            else:
                                color_msg.data_bool = False
                        self.send_request(self.set_target_client, color_msg)
                    # Enable sorting. 开启分拣
                    start_msg = SetBool.Request()
                    start_msg.data = True
                    self.send_request(self.start_client, start_msg)
                if 'response' in result:
                    msg.data = result['response']
            else: # No corresponding action, just respond. 没有对应的行为，只回答
                msg.data = self.llm_result
                self.tts_text_pub.publish(msg)
                self.action_finish = True
                self.llm_result = ''
        else:
            time.sleep(0.01)
    if self.play_audio_finish and self.action_finish:
        self.play_audio_finish = False
        self.action_finish = False
        msg = SetBool.Request()
        msg.data = True
        self.send_request(self.awake_client, msg)
    rclpy.shutdown()

```

主处理循环，解析大模型结果，提取颜色分拣动作（如 `color_sorting('red')`），通过正则提取目标颜色，设置分拣目标并启动分拣；发布 TTS 回复，完成后重新启用唤醒功能。

◆ main函数

```

def main():
    node = LLMColorSorting('llm_color_sorting')
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    executor.spin()
    node.destroy_node()

```

节点入口，创建 `LLMColorSorting` 实例，启动多线程执行器处理回调，运行节点并清理资源。

4. 垃圾分类

本节课的大模型属于线上大模型，需要让我们使用的主控保持网络通畅（建议使用有线网络进行连接）。

4.1. 玩法简介

程序开始运行，“AI 语音交互盒”播报“准备就绪”后。说出指定的唤醒词“hello hiwonder”激活“AI 语音交互盒”，“AI 语音交互盒”会回应“我在”。然后即可通过语音控制 ArmPi Ultra，例如“清理厨余垃圾”，终端会打印出识别到的语音，“AI 语音交互盒”会播报思考后生成的回答，并执行对应的玩法。

4.2. 准备工作

4.2.1. 确认 WonderEcho Pro 固件

WonderEcho Pro 出厂默认的唤醒词是“hello, hiwonder”。如果想要更换唤醒词为“小幻小幻”，可以参考教程目录下“11.语音控制课程\语音基础课程\02 固件烧录”及同目录下的“固件烧录视频”。

假设之前已经根据以上方法，烧录过其他固件（例如“小幻小幻”唤醒词固件），则需要使用对应的唤醒词进行唤醒。

下面的课程示例默认以出厂唤醒词“hello, hiwonder”为例进行说明。

4.2.2. 配置大模型相关 key

程序默认不配置大模型相关密钥，在开启玩法前需要可以参考本章节“[大模型 API-KEY 获取及配置](#)”完成对相关密钥的配置，此步骤不可跳过，不然影响大模型相关玩法的体验。

4.2.3. 配置网络

注意:本节课的大模型是属于线上大模型，在开始前需要准备一根网线用于连接网络或者将机器人切换成局域网模式。

在此玩法模式下，机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。网络的具体配置方法有以下两种方式：

- 通过手机 APP 进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“手机 APP 控制/连接模式介绍（选看）”。

- 通过 VNC 远程连接到机器人修改相关配置文件进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“开发环境搭建及配置/局域网模式连接”。

4.3. 玩法步骤

i 1. 指令输入需严格区分大小写及空格。

2. 机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。

- 1) 点击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。
- 2) 输入指令，关闭手机 app 自启服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

- 3) 输入指令，按下回车键运行垃圾分类的玩法。

```
ros2 launch large_models llm_waste_classification.launch.py
```

```
ros2 launch large_models llm_waste_classification.launch.py
```

4) 当命令行出现如下图输出并播报“准备就绪”，表示“AI 语音交互盒”完成初始化，此时可以说出唤醒词“hello hiwonder”。

```
[tts_node-12] [INFO] [1754298926.158559557] [tts_node]: start
[vocal_detect-10] [INFO] [1754298926.800849997] [vocal_detect]: Rate:48000 Size:3200 Channel:2 Device:hw:2,0
[vocal_detect-10] [INFO] [1754298926.856092760] [vocal_detect]: start
[agent_process-11] [INFO] [1754298927.063444235] [agent_process]: start
[agent_process-11] [INFO] [1754298927.943117559] [agent_process]: set model
[agent_process-11] [INFO] [1754298928.015443583] [agent_process]: set prompt
[yolov8_node-8] [INFO] [1754298928.053525215] [yolov8]: start yolov8 detect
[llm_waste_classification-13] [INFO] [1754298929.578117412] [llm_waste_classification]: start
```

5) 当命令行出现如下图输出，“AI 语音交互盒”播报“我在”，表示“AI 语音交互盒”已被唤醒激活。此时开始录制用户的指令。

```
[vocal_detect-10] [INFO] [1754299720.110415456] [vocal_detect]: asr...
[vocal_detect-10] [INFO] [1754299720.148662395] [vocal_detect]: Recording.....
```

- 6) 我们可以根据程序出现的任务示例里面的输入提示信息说出命令。

```
[llm_waste_classification-13] ## 任务示例
[llm_waste_classification-13] 输入：将厨余垃圾拿走
[llm_waste_classification-13] 输出：{'action': "garbage_classification('food_waste')", 'response': '收到，我马上收拾'}
[llm_waste_classification-13] 输入：把香蕉皮拿走
[llm_waste_classification-13] 输出：{'action': "garbage_classification('food_waste')", 'response': '好的，香蕉皮属于厨余垃圾'}
[llm_waste_classification-13] 输入：清理有害垃圾
[llm_waste_classification-13] 输出：{'action': "garbage_classification('hazardous_waste')", 'response': '有害垃圾必须清除'}
[llm_waste_classification-13] 输入：将可回收垃圾找出来
[llm_waste_classification-13] 输出：{'action': "garbage_classification('recyclable_waste')", 'response': '这可难不倒我'}
[llm_waste_classification-13] 输入：分拣其他垃圾
[llm_waste_classification-13] ### 期望回应：{'action': "garbage_classification('residual_waste')", 'response': '收到，收到'}
[llm_waste_classification-13] 输入：先把厨余垃圾拿走，再把有害垃圾清理了
[llm_waste_classification-13] 输出：{'action': "garbage_classification('food_waste', 'hazardous_waste')", 'response': '垃圾清理，马上执行'}
[llm_waste_classification-13] 输入：只留下可回收垃圾，把其余都拿走
[llm_waste_classification-13] 输出：{'action': "garbage_classification('food_waste', 'hazardous_waste', 'residual_waste')", 'response': '没问题'}
```

7) 例如，我们将对应的垃圾标签摆放到摄像头的面前，说出口令“香蕉皮拿走”。



8) 当命令行出现如下图输出，表示“AI 语音交互盒”将识别到的语音打印出来。

```
[vocal_detect-10] [INFO] [1754299720.110415456] [vocal_detect]: asr...
[vocal_detect-10] [INFO] [1754299720.148662395] [vocal_detect]: Recording.....
[vocal_detect-10] [INFO] [1754299721.678306881] [vocal_detect]: 把香蕉
[vocal_detect-10] [INFO] [1754299721.681152929] [vocal_detect]: 把香蕉
[vocal_detect-10] [INFO] [1754299724.272906578] [vocal_detect]: 把香蕉皮拿
[vocal_detect-10] [INFO] [1754299724.449964185] [vocal_detect]: 把香蕉皮拿走。
[vocal_detect-10] [INFO] [1754299724.453776126] [vocal_detect]: RecognitionCallback sentence end, request_id:04c5e302b26e41c98bdb2b4db89d94bc, usage: {'duration': 1}
[vocal_detect-10] [INFO] [1754299724.565898453] [vocal_detect]: RecognitionCallback completed.
```

9) 当命令行出现如下图输出，表示成功调用云端语音大模型的语音识别服务，对用户指令音频进行解析，“publish_asr_result”中为解析结果。

```
[vocal_detect-10] [INFO] [1754299724.613155889] [vocal_detect]: Done recording
[vocal_detect-10] [INFO] [1754299725.314040162] [vocal_detect]: publish asr result:把香蕉皮拿走。
[agent_process-11] [INFO] [1754299725.318783976] [agent_process]: thinking...
```

10) 当命令行出现如下图输出，表示成功调用云端大语言大模型，对用户指令就行思考，并给出语言回答“response”并设计符合用户指令语义的动作。

i 回答为大模型自动生成，只保证回复的语义准确，回复的文字编排具有随机性。

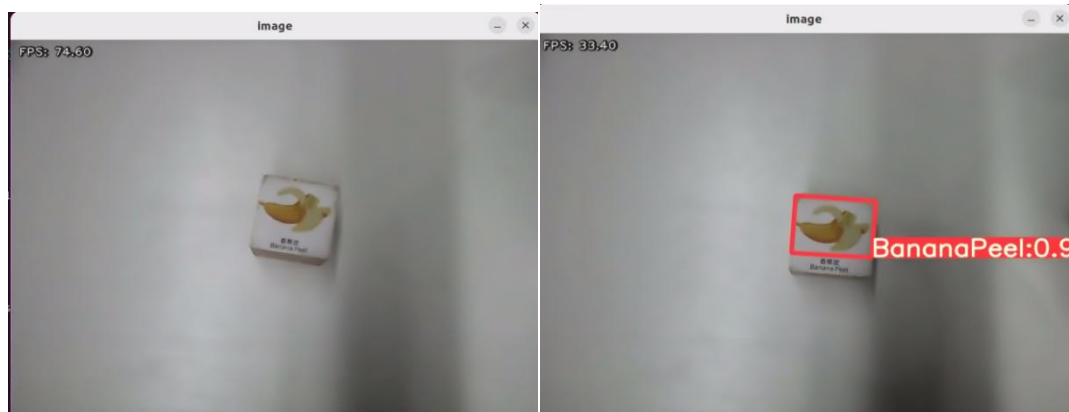
```
[agent_process-11] [INFO] [1754299730.250373041] [agent_process]: publish llm result: ``json
[agent_process-11] {'action': "garbage_classification('food_waste')", 'response': '香蕉皮归我，厨余垃圾妥妥的'}
[agent_process-11] ``
```

11) 当命令行出现如下图输出，表示一轮对话交互已经结束，您可以参考步骤 4)，重新以唤醒词唤醒“AI 语音交互盒”，开始新一轮对话交互。

```
[tts_node-12] [INFO] [1754299730.974582199] [tts_node]: Speech synthesizer is opened.
[tts_node-12] [INFO] [1754299733.772049774] [tts_node]: Speech synthesizer is completed.
[tts_node-12] [INFO] [1754299733.792586276] [tts_node]: Speech synthesizer is closed.
[vocal_detect-10] [INFO] [1754299733.862892225] [vocal_detect]: enable_wakeup
```

4.4. 实现效果

玩法开启后，我们可以自由组织语言，例如：“把香蕉拿走”，控制 ArmPi Ultra 自动抓取前方视线范围内的香蕉垃圾方块（或者厨余垃圾）并放置到机械臂的右侧。



4.5. 程序分析

4.5.1. Launch 文件分析

程序路径：

```
/home/ubuntu/ros2_ws/src/large_models/large_models/llm_waste_classification.launch.py
```

◆ launch_setup函数

```

def launch_setup(context):
    mode = LaunchConfiguration('mode', default=1)
    mode_arg = DeclareLaunchArgument('mode', default_value=mode)

    large_models_package_path = get_package_share_directory('large_models')

    # 立即启动 waste_classification_launch
    waste_classification_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(large_models_package_path, 'launch/waste_classification.launch.py')),
    )

    # 在5秒后启动 large_models_launch 和 llm_waste_classification_node
    delayed_launch = TimerAction(
        period=5.0, # 延迟时间为5秒
        actions=[
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(
                    os.path.join(large_models_package_path, 'launch/start.launch.py')),
                launch_arguments={'mode': mode}.items(),
            ),
            Node(
                package='large_models',
                executable='llm_waste_classification',
                output='screen',
            ),
        ]
    )

    return [mode_arg,
            waste_classification_launch, # 立即启动
            delayed_launch,             # 5秒后启动
    ]

```

立即加载 large_models 功能包的 launch/waste_classification.launch.py，启动垃圾分类的基础节点（如垃圾识别、类别判断等核心功能），作为系统的底层支撑。延迟启动大模型与控制节点，通过 TimerAction 设置 5 秒延迟，确保基础节点初始化完成后，再启动依赖的上层组件，加载 large_models 功能包的 launch/start.launch.py，启动大模型服务，并传入 mode 参数控制其运行模式（如不同的分类逻辑或交互方式）。启动 large_models 功能包中的 llm_waste_classification 可执行文件，作为大模型与垃圾分类逻辑的桥梁，将大模型的决策（如垃圾类别判断）转换为具体的分拣动作指令，日志输出到屏幕。

◆ generate_launch_description函数

```

def generate_launch_description():
    return LaunchDescription([
        OpaqueFunction(function = launch_setup)
    ])

```

创建并返回 LaunchDescription 对象，通过 OpaqueFunction 调用 launch_setup 函数，作为 ROS 2 启动文件的标准入口。

◆ 主函数

```
if __name__ == '__main__':  
    # 创建一个LaunchDescription对象  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务，将启动描述添加到启动服务并运行，实现手动启动整个系统的功能

4.5.2. Python 文件分析

程序路径：

/home/ubuntu/ros2_ws/src/large_models/large_models/llm_waste_classification.py

◆ 定义提示词模板（PROMPT）

```
## 任务示例  
输入：将厨余垃圾拿走  
输出：{'action': "garbage_classification('food_waste')", 'response': '收到，我马上收拾'}  
输入：把香蕉皮拿走  
输出：{'action': "garbage_classification('food_waste')", 'response': '好的，香蕉皮属于厨余垃圾'}  
输入：清理有害垃圾  
输出：{'action': "garbage_classification('hazardous_waste')", 'response': '有害垃圾必须清除'}  
输入：将可回收垃圾找出来  
输出：{'action': "garbage_classification('recyclable_waste')", 'response': '这可难不倒我'}  
输入：分拣其他垃圾  
期望回应：{'action': "garbage_classification('residual_waste')", 'response': '收到，收到'}  
输入：先把厨余垃圾拿走，再把有害垃圾清理了  
输出：{'action': "garbage_classification('food_waste', 'hazardous_waste')", 'response': '垃圾清理，马上执行'}  
输入：只留下可回收垃圾，把其他都拿走  
输出：{'action': "garbage_classification('food_waste', 'hazardous_waste', 'residual_waste')", 'response': '没问题'}  
...
```

◆ 初始化

```

class LLMWasteClassification(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name)

        self.action = []
        self.llm_result = ''
        self.action_finish = False
        self.play_audio_finish = False
        # self.llm_result = '{"action": "garbage_classification(\'food_waste\', \'hazardous_waste\', \'residual_waste\', \'recyclable_waste\')", "response": "收到，我马上收拾"}'
        self.running = True

        timer_cb_group = ReentrantCallbackGroup()
        self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
        self.create_subscription(String, 'agent_process/result', self.llm_result_callback, 1)
        self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_finish_callback, 1, callback_group=timer_cb_group)
        self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callback, 1)
        self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wakeup')
        self.awake_client.wait_for_service()

        self.set_model_client = self.create_client(SetModel, 'agent_process/set_model')
        self.set_model_client.wait_for_service()

        self.set_prompt_client = self.create_client(SetString, 'agent_process/set_prompt')
        self.set_prompt_client.wait_for_service()

        self.enter_client = self.create_client(Trigger, 'waste_classification/enter')
        self.enter_client.wait_for_service()

        self.start_client = self.create_client(SetBool, 'waste_classification/enable_transport')
        self.start_client.wait_for_service()

        self.set_target_client = self.create_client(SetStringList, 'waste_classification/set_target')
        self.set_target_client.wait_for_service()

        self.timer = self.create_timer(0.0, self.init_process, callback_group=timer_cb_group)

```

初始化节点，配置状态变量（如动作列表、大模型结果等），创建发布者（TTS 文本）、订阅者（大模型结果、TTS 完成信号、唤醒信号）和服务客户端（大模型配置、垃圾分拣控制等），启动初始化定时器。

◆ get_node_state方法

```

def get_node_state(self, request, response):
    return response

```

服务回调函数，返回空响应，用于节点初始化完成确认。

◆ init_process方法

```

def init_process(self):
    self.timer.cancel()

    msg = SetModel.Request()
    msg.model = llm_model
    msg.model_type = 'llm'
    msg.api_key = api_key
    msg.base_url = base_url
    self.send_request(self.set_model_client, msg)

    msg = SetString.Request()
    msg.data = PROMPT
    self.send_request(self.set_prompt_client, msg)

    self.send_request(self.enter_client, Trigger.Request())

    speech.play_audio(start_audio_path)
    threading.Thread(target=self.process, daemon=True).start()
    self.create_service(Empty, '~/init_finish', self.get_node_state)
    self.get_logger().info('\033[1;32m%s\033[0m' % 'start')
    self.get_logger().info('\033[1;32m%s\033[0m' % PROMPT)

```

节点初始化流程，取消定时器，配置大模型参数（模型类型、API 密钥等），发送进入垃圾分类服务请求，启动处理线程，发布初始化完成日志。

◆ send_request方法

```

def send_request(self, client, msg):
    future = client.call_async(msg)
    while rclpy.ok():
        if future.done() and future.result():
            return future.result()

```

通用服务调用工具，发送服务请求并阻塞等待结果，确保服务通信完成（如配置大模型、控制分拣）。

◆ wakeup_callback方法

```

def wakeup_callback(self, msg):
    if msg.data and self.llm_result:
        self.send_request(self.enter_client, Trigger.Request())

```

唤醒信号订阅回调，收到唤醒信号时重新发送进入垃圾分类服务请求，重置分拣状态。

◆ llm_result_callback方法

```
def llm_result_callback(self, msg):  
    self.llm_result = msg.data
```

大模型结果订阅回调，接收大模型生成的指令（含 action 和 response），保存到 llm_result 供处理。

◆ play_audio_finish_callback方法

```
def play_audio_finish_callback(self, msg):  
    self.play_audio_finish = msg.data
```

TTS 播放完成信号订阅回调，设置 play_audio_finish 标志，同步动作执行与语音播放。

◆ process方法

```

def process(self):
    while self.running:
        if self.llm_result:
            msg = String()
            if 'action' in self.llm_result: # 如果有对应的行为返回那么就提取处理
                self.get_logger().info(self.llm_result)
                result = eval(self.llm_result[self.llm_result.find('{'):self.llm_result.find('}') + 1])
                if 'action' in result:
                    self.get_logger().info(result['action'])
                    text = result['action']
                    # 使用正则表达式提取括号中的所有字符串
                    pattern = r"garbage_classification\(((\[\"'][^\"']+\[\"'](?:\s*\s*\[\"'][^\"']+\[\"'])*\))\)"
                    # 使用re.search找到匹配的结果
                    match = re.search(pattern, text)
                    # 提取结果
                    if match:
                        # 获取所有的参数部分（括号内的内容）
                        params = match.group(1)
                        # 使用正则拆分出各个字符串参数
                        objects = re.findall(r"\"'([^\"]+)[^\"]\"", params)
                        # self.get_logger().info(str(objects))
                        # 开启识别
                        msgs = SetStringList.Request()
                        msgs.data = objects
                        res = self.send_request(self.set_target_client, msgs)
                        if res.success:
                            self.get_logger().info('\033[1;32m%s\033[0m' % 'set object success')
                        else:
                            self.get_logger().info('\033[1;32m%s\033[0m' % 'set object fail')
                        start_msg = SetBool.Request()
                        start_msg.data = True
                        # 开启分拣
                        self.send_request(self.start_client, start_msg)
                    if 'response' in result:
                        msg.data = result['response']
                else: # 没有对应的行为，只回答
                    msg.data = self.llm_result
                self.tts_text_pub.publish(msg)
                self.action_finish = True
                self.llm_result = ''
            else:
                time.sleep(0.01)
        if self.play_audio_finish and self.action_finish:
            self.play_audio_finish = False
            self.action_finish = False
            msg = SetBool.Request()
            msg.data = True
            self.send_request(self.awake_client, msg)
    rclpy.shutdown()

```

主处理循环，解析大模型结果，提取垃圾分类动作（如 `garbage_classification('food_waste')`），通过正则提取目标垃圾类型，设置分拣目标并启动分拣；发布 TTS 回复，完成后重新启用唤醒功能。

◆ main函数

```

def main():
    node = LLMWasteClassification('llm_waste_classification')
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    executor.spin()
    node.destroy_node()

```

节点入口，创建 `LLMWasteClassification` 实例，启动多线程执行器处理回调，运行节点并清理资源。

5. 三维物体分拣

本节课的大模型属于线上大模型，需要让我们使用的主控保持网络通畅（建议使用有线网络进行连接）。

5.1. 玩法简介

程序开始运行，“AI 语音交互盒”播报“准备就绪”后。

说出指定的唤醒词“hello hiwonder”激活“AI 语音交互盒”，“AI 语音交互盒”会回应“我在”。然后即可通过语音控制 ArmPi Ultra，例如“将圆柱体拿走”，终端会打印出识别到的语音，“AI 语音交互盒”会播报思考后生成的回答，并执行对应的玩法。

5.2. 准备工作

5.2.1. 确认 WonderEcho Pro 固件

WonderEcho Pro 出厂默认的唤醒词是“hello, hiwonder”。如果想要更换唤醒词为“小幻小幻”，可以参考教程目录下“11.语音控制课程\语音基础课程\02 固件烧录”及同目录下的“固件烧录视频”。

假设之前已经根据以上方法，烧录过其他固件（例如“小幻小幻”唤醒词固件），则需要使用对应的唤醒词进行唤醒。

下面的课程示例默认以出厂唤醒词“hello, hiwonder”为例进行说明。

5.2.2. 配置大模型相关 key

程序默认不配置大模型相关密钥，在开启玩法前需要可以参考本章节“[大模型 API-KEY 获取及配置](#)”完成对相关密钥的配置，此步骤不可跳过，不然影响大模型相关玩法的体验。

5.2.3. 配置网络

注意:本节课的大模型是属于线上大模型，在开始前需要准备一根网线用于连接网络或者将机器人切换成局域网模式。

在此玩法模式下，机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。网络的具体配置方法有以下两种方式：

- 通过手机 APP 进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“手机 APP 控制/连接模式介绍（选看）”。

- 通过 VNC 远程连接到机器人修改相关配置文件进行网络配置，具体的路径如下。

“教程资料/ArmPi Ultra 使用入门手册”文档内的“开发环境搭建及配置/局域网模式连接”。

5.3. 玩法步骤

i 1. 指令输入需严格区分大小写及空格。

2. 机器人必须联网，配置为 STA（局域网）模式或 AP（直连）模式下连接网线。

- 1) 点击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。
- 2) 输入指令，关闭手机 app 自启服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

- 3) 输入指令，按下回车键运行三维物体分拣的玩法。

```
ros2 launch large_models llm_3d_object_sorting.launch.py
```

```
ros2 launch large_models llm_3d_object_sorting.launch.py
```

4) 当命令行出现如下图输出并播报“准备就绪”，表示“AI 语音交互盒”完成初始化，此时可以说出唤醒词“hello hiwonder”。

```
[vocal_detect-8] [INFO] [1741427315.702633242] [vocal_detect]: start
[tts_node-10] [INFO] [1741427316.858147378] [tts_node]: start
[agent_process-9] [INFO] [1741427317.152110653] [agent_process]: start
[agent_process-9] [INFO] [1741427317.402451747] [agent_process]: set model
[agent_process-9] [INFO] [1741427317.515325436] [agent_process]: set prompt
[shape_recognition-7] [INFO] [1741427317.553281558] [shape_recognition]: enter shape recognition
[llm_3d_object_sorting-11] [INFO] [1741427319.500577837] [llm_object_sorting]: start
```

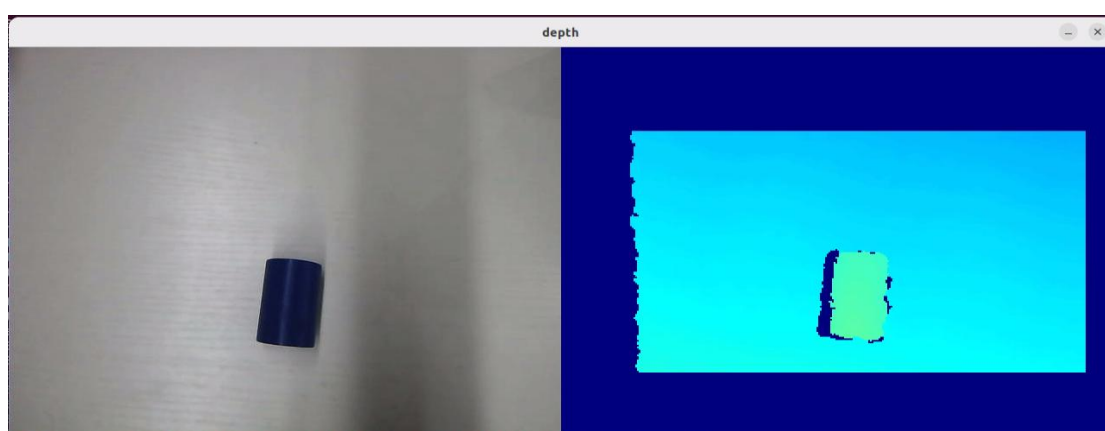
5) 当命令行出现如下图输出，“AI 语音交互盒”播报“我在”，表示“AI 语音交互盒”已被唤醒激活。此时开始录制用户的指令。

```
[vocal_detect-7] [INFO] [1741419606.799872983] [vocal_detect]: asr...
[vocal_detect-7] [INFO] [1741419606.805699617] [vocal_detect]: Recording.....
[vocal_detect-7] [INFO] [1741419608.378863034] [vocal_detect]:
```

6) 我们可以根据程序出现的**任务示例**里面的**输入**提示信息说出命令。

```
##任务示例
输入：分拣圆柱体
输出：{"action": "object_sorting('cylinder')", "response": "好的，开始分拣圆柱体"}
输入：将长方体拿走
输出：{"action": "object_sorting('cuboid')", "response": "长方体走你"}
输入：只留下球体，其他拿走
输出：{"action": "object_sorting('cylinder', 'cuboid')", "response": "好的，马上执行"}
输入：分拣物体
输出：{"action": "object_sorting('cylinder', 'cuboid', 'sphere')", "response": "没问题"}
```

7) 例如，我们将道具摆放到摄像头的面前，说出口令“将圆柱体拿走”。



8) 当命令行出现如下图输出，表示“AI 语音交互盒”将识别到的语音打印出来。

```
[vocal_detect]: 把
[vocal_detect]: 把圆柱
[vocal_detect]: 把圆柱体拿
[vocal_detect]: 把圆柱体拿走。
[vocal_detect]: RecognitionCallback sentence end, request_id:0e6cf02b4eba482ca4d7745564445fae, usage:{'duration': 3}
```

9) 当命令行出现如下图输出，表示成功调用云端语音大模型的语音识别服务，对用户指令音频进行解析，“publish_asr_result”中为解析结果。

```
RecognitionCallback completed.
Done recording
publish asr result:把圆柱体拿走。
]: thinking...
```

10) 当命令行出现如下图输出，表示成功调用云端大语言大模型，对用户指令就行思考，并给出语言回答“response”并设计符合用户指令语义的动作。

i 回答为大模型自动生成，只保证回复的语义准确，回复的文字编排具有随机性。

```
[agent_process]: publish llm result:{"json": "cylinder'")", "response": "圆柱体再见，挥挥手不带走一丝尘垢！"}]
```

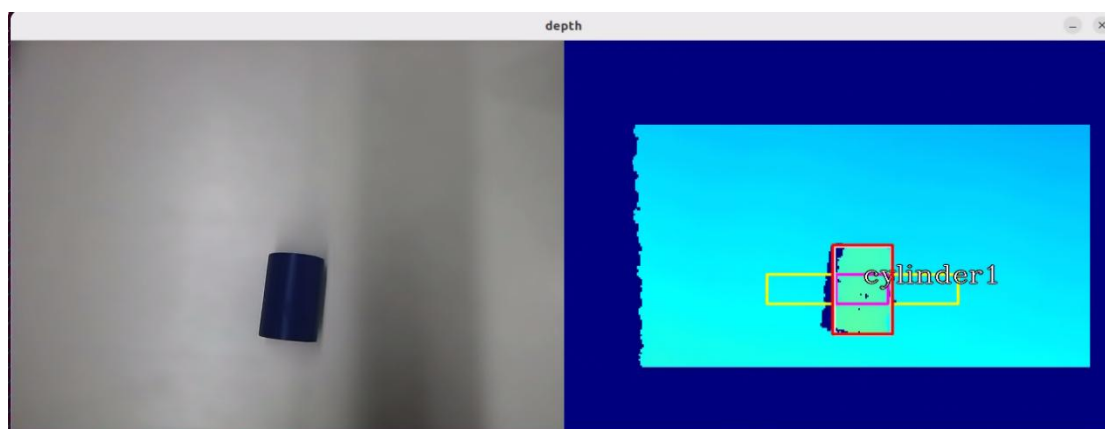
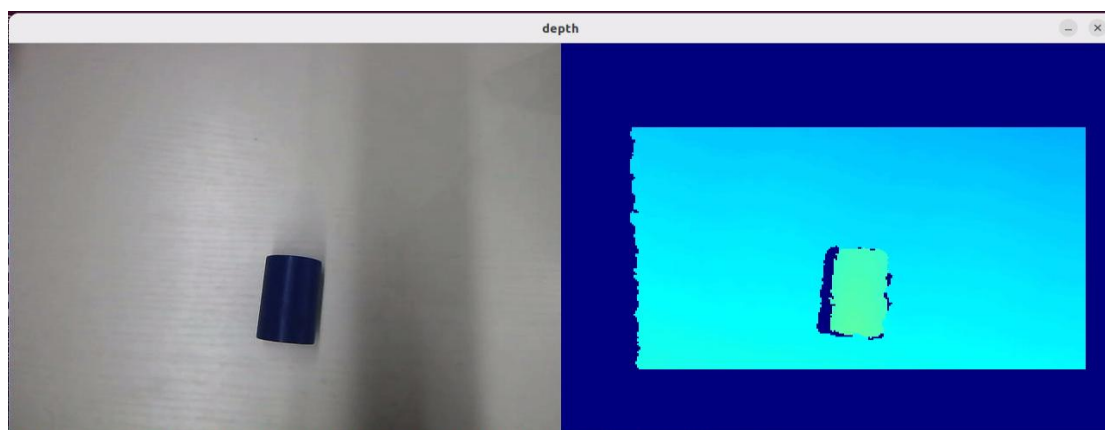
11) 当命令行出现如下图输出，表示一轮对话交互已经结束，您可以参考步骤 4)，重新以唤醒词唤醒“AI 语音交互盒”，开始新一轮对话交互。

```
[agent_process-9] [tts_node-9] [INFO] [1741419615.424822235] [tts_node]: Speech synthesizer is opened.  
[tts_node-9] [INFO] [1741419618.390946900] [tts_node]: Speech synthesizer is completed.  
[tts_node-9] [INFO] [1741419618.395256042] [tts_node]: Speech synthesizer is closed.  
[vocal_detect-7] [INFO] [1741419618.402115169] [vocal_detect]: enable_wakeup
```

如需关闭此玩法，只需要在终端界面中按下“Ctrl+C”。如果关闭失败，可多次按下。

5.4. 实现效果

玩法开启后，我们可以自由组织语言，例如：“将圆柱体拿走”，控制 ArmPi Ultra 自动抓取前方视线范围内的圆柱体方块并放置到机械臂的左侧。



5.5. 程序分析

5.5.1. Launch 文件分析

程序路径：

`/home/ubuntu/ros2_ws/src/large_models/large_models/llm_3d_object_sorting.launch.py`

◆ launch_setup函数

```
def launch_setup(context):
    mode = LaunchConfiguration('mode', default=1)
    mode_arg = DeclareLaunchArgument('mode', default_value=mode)

    # 获取包路径
    large_models_package_path = get_package_share_directory('large_models')

    object_sorting_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(large_models_package_path, 'launch/3d_object_sorting.launch.py')),
    )

    # 在5秒后启动 large_models_launch 和 llm_3d_object_sorting_node
    delayed_launch = TimerAction(
        period=5.0, # 延迟时间为5秒
        actions=[
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(
                    os.path.join(large_models_package_path, 'launch/start.launch.py')),
                launch_arguments={'mode': mode}.items(),
            ),
            Node(
                package='large_models',
                executable='llm_3d_object_sorting',
                output='screen',
            ),
        ],
    )

    return [mode_arg,
            object_sorting_launch, # 立即启动 object_sorting_launch
            delayed_launch,       # 启动后延迟的部分
    ]
```

立即加载 large_models 功能包的 launch/3d_object_sorting.launch.py，启动 3D 物体分拣的基础节点（如 3D 物体识别、空间定位、类别判断等核心功能），作为系统的底层支撑，提供三维空间中的物体信息。延迟启动大模型与控制节点，通过 TimerAction 设置 5 秒延迟，确保基础节点（如 3D 识别模块）初始化完成并稳定输出数据后，再启动依赖的上层组件，加载 large_models 功能包的 launch/start.launch.py，启动大模型

服务，并传入 mode 参数控制其运行模式（如不同的 3D 分拣决策逻辑、交互方式等）。启动 large_models 功能包中的 llm_3d_object_sorting 可执行文件，作为大模型与 3D 物体分拣逻辑的桥梁，将大模型的决策（如基于 3D 位置的抓取策略）转换为具体的机械臂动作指令，日志输出到屏幕。

◆ generate_launch_description函数

```
def generate_launch_description():  
    return LaunchDescription([  
        OpaqueFunction(function = launch_setup)  
    ])
```

创建并返回 LaunchDescription 对象，通过 OpaqueFunction 调用 launch_setup 函数，作为 ROS 2 启动文件的标准入口。

◆ 主函数

```
if __name__ == '__main__':  
    # 创建一个LaunchDescription对象  
    ld = generate_launch_description()  
  
    ls = LaunchService()  
    ls.include_launch_description(ld)  
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务，将启动描述添加到启动服务并运行，实现手动启动整个系统的功能

5.5.2. Python 文件分析

程序路径：

/home/ubuntu/ros2_ws/src/large_models/large_models/llm_3d_object_sorting.py

◆ 定义提示词模板（PROMPT）

```

## 任务示例
输入：分拣圆柱体
输出：{'action': "object_sorting('cylinder')", 'response': '好的，开始分拣圆柱体'}
输入：将长方体拿走
输出：{'action': "object_sorting('cuboid')", 'response': '长方体走你'}
输入：只留下球体，其他拿走
输出：{'action': "object_sorting('cylinder','cuboid')", 'response': '好的，马上执行'}
输入：分拣物体
输出：{'action': "object_sorting('cylinder', 'cuboid','sphere')", 'response': '没问题'}
...

```

◆ 初始化

```

class LLMObjectSorting(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name)

        self.action = []
        self.llm_result = ''
        self.action_finish = False
        self.play_audio_finish = False
        # self.llm_result = '{"action": "object_sorting(\'cylinder\', \'cuboid\', \'sphere\')", "response": "收到，我马上收拾"}'
        self.running = True

        timer_cb_group = ReentrantCallbackGroup()
        self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
        self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_finish_callback, 1, callback_group=timer_cb_group)
        self.create_subscription(String, 'agent_process/result', self.llm_result_callback, 1)
        self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callback, 1)

        self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wakeup')
        self.awake_client.wait_for_service()

        self.set_model_client = self.create_client(SetModel, 'agent_process/set_model')
        self.set_model_client.wait_for_service()

        self.set_prompt_client = self.create_client(SetString, 'agent_process/set_prompt')
        self.set_prompt_client.wait_for_service()

        self.enter_client = self.create_client(Trigger, 'shape_recognition/enter')
        self.enter_client.wait_for_service()

        self.start_client = self.create_client(SetBool, 'shape_recognition/set_running')
        self.start_client.wait_for_service()

        self.set_target_client = self.create_client(SetStringList, 'shape_recognition/set_shape')
        self.set_target_client.wait_for_service()

        self.timer = self.create_timer(0.0, self.init_process, callback_group=timer_cb_group)

```

初始化节点，配置状态变量（如动作列表、大模型结果等），创建发布者（TTS 文本）、订阅者（大模型结果、TTS 完成信号、唤醒信号）和服务客户端（大模型配置、物体分拣控制等），启动初始化定时器。

◆ get_node_state方法

```

def get_node_state(self, request, response):
    return response

```

服务回调函数，返回空响应，用于节点初始化完成确认。

◆ init_process方法

```

def init_process(self):
    self.timer.cancel()

    msg = SetModel.Request()
    msg.model = llm_model
    msg.model_type = 'llm'
    msg.api_key = api_key
    msg.base_url = base_url
    self.send_request(self.set_model_client, msg)

    msg = SetString.Request()
    msg.data = PROMPT
    self.send_request(self.set_prompt_client, msg)

    self.send_request(self.enter_client, Trigger.Request())

    speech.play_audio(start_audio_path)
    threading.Thread(target=self.process, daemon=True).start()
    self.create_service(Empty, '~/init_finish', self.get_node_state)
    self.get_logger().info('\033[1;32m%s\033[0m' % 'start')
    self.get_logger().info('\033[1;32m%s\033[0m' % PROMPT)

```

节点初始化流程，取消定时器，配置大模型参数（模型类型、API 密钥等），发送进入物体分拣服务请求，启动处理线程，发布初始化完成日志。

◆ send_request方法

```

def send_request(self, client, msg):
    future = client.call_async(msg)
    while rclpy.ok():
        if future.done() and future.result():
            return future.result()

```

通用服务调用工具，发送服务请求并阻塞等待结果，确保服务通信完成（如配置大模型、控制分拣）。

◆ wakeup_callback方法

```

def wakeup_callback(self, msg):
    if msg.data and self.llm_result:
        self.send_request(self.enter_client, Trigger.Request())

```

唤醒信号订阅回调，收到唤醒信号时重新发送进入物体分拣服务请求，重置分拣状态。

◆ llm_result_callback方法

```
def llm_result_callback(self, msg):
    self.llm_result = msg.data
```

大模型结果订阅回调，接收大模型生成的指令（含 action 和 response），保存到 llm_result 供处理。

◆ play_audio_finish_callback方法

```
def play_audio_finish_callback(self, msg):
    self.play_audio_finish = msg.data
```

TTS 播放完成信号订阅回调, 设置 play_audio_finish 标志, 同步动作执行与语音播放。

◆ process方法

```
def process(self):
    while self.running:
        if self.llm_result:
            msg = String()
            if 'action' in self.llm_result: # If there is a corresponding action returned, extract and handle it. 如果有对应的行为返回那么就提取处理
                result = eval(self.llm_result[self.llm_result.find('('):self.llm_result.find(')') + 1])
                if 'action' in result:
                    text = result['action']
                    # Use regular expressions to extract all strings inside. 使用正则表达式提取括号中的所有字符串
                    pattern = r"object_sorting\(((['\"])([^\"]|'')*(?!\s*,\s*['\"])([^\"]|'')*(?!\s*\))\)"
                    # Use re.search to find the matching result. 使用re.search找到匹配的结果
                    match = re.search(pattern, text)
                    # Extract the result. 提取结果
                    if match:
                        # Retrieve all argument parts (the content inside the parentheses). 获取所有的参数部分（括号内的内容）
                        params = match.group(1)
                        # Use regular expressions to split the string arguments. 使用正则拆分出各个字符串参数
                        objects = re.findall(r"['\"]([^\"]|'')*['\"]", params)
                        # self.get_logger().info(str(objects))
                        # Enable recognition. 开启识别
                        msgs = SetStringList.Request()
                        msgs.data = objects
                        res = self.send_request(self.set_target_client, msgs)
                        if res.success:
                            self.get_logger().info('\033[1;32m%s\033[0m % 'set object success')
                        else:
                            self.get_logger().info('\033[1;32m%s\033[0m % 'set object fail')
                        start_msg = SetBool.Request()
                        start_msg.data = True
                        # Enable sorting. 开启分拣
                        self.send_request(self.start_client, start_msg)
                    if 'response' in result:
                        msg.data = result['response']
                else: # No corresponding action, just respond. 没有对应的行为，只回答
                    msg.data = self.llm_result
                    self.tts_text_pub.publish(msg)
                    self.action_finish = True
                    self.llm_result = ''
            else:
                time.sleep(0.01)
        if self.play_audio_finish and self.action_finish:
            self.play_audio_finish = False
            self.action_finish = False
            msg = SetBool.Request()
            msg.data = True
            self.send_request(self.awake_client, msg)
    rcplpy.shutdown()
```

主处理循环，解析大模型结果，提取物体分拣动作（如 `object_sorting('cylinder')`），通过正则提取目标物体类型，设置分拣目标并启动分拣；发布 TTS 回复，完成后重新启用唤醒功能。

◆ main函数

```
def main():  
    node = LLMObjectSorting('llm_object_sorting')  
    executor = MultiThreadedExecutor()  
    executor.add_node(node)  
    executor.spin()  
    node.destroy_node()
```

节点入口，创建 `LLMObjectSorting` 实例，启动多线程执行器处理回调，运行节点并清理资源。